Documentation

To start the raytracer, first open up the scene in Blender. Use the Export.py file in the Blender folder to get the ASCII scene.txt file. If there are textures in the scene, make sure that it is in an accessible path.

To run the code, type the following:

```
mkdir -p build
cd build
cmake ..
make
```

Then type: ./raytracer --scene scene.txt --out output.ppm

scene.txt is the name of the ASCII file from Blender, and output.ppm is the name of the final rendered image. If the scene.txt is not in the current directory as the code, you can also input the path as shown below:

./raytracer --scene '/Users/stephilyou/Documents/Fall Quarter 2025/Computer Graphics/Coursework/Blend/scene.txt' --out output.ppm

This will render using the acceleration hierarchy and antialiasing. If you want to turn those features off, you can enter --no-bvh or --no-aa as shown below:

./raytracer --scene scene.txt --out output.ppm --no-bvh --no-aa

For more information, please read the README in the Code folder.

Implemented Features

Feature	Percentage Completed
Blender Exporter	100%
Camera Space Transformation	100%
Image Read/Write	100%
Ray Intersection	100%

Acceleration Hierarchy	100%
Whitted-style Ray Tracing	100%
Antialiasing	100%
Textures	100%
System Integration	100%
Distributed Raytracing	0%
Lens Effect	0%

Blender Exporter

My Blender exporter python file has 4 functions: get_material_properties(), get_texture_path_from_material(), convert_jpg_to_ppm(), and export_to_ascii(). The get_material_properties() uses the Blender scene to extract the Blinn-Phong properties from the objects. The get_texture_path_from_material() gets the path of the texture file from the material's Image Texture node. The convert_jpg_to_ppm() uses Blender's image API to convert any JPG texture image files into either P6 or P3 PPM format. The export_to_ascii() function goes through all objects in the scene to extract the appropriate information needed to render and writes it into a txt file. I did not encounter any difficulties with this feature. I used the coding assistant for the Blinn-Phong properties and wrote code by myself for the exporting section. The following image demonstrates the resulting txt file using my Blender exporter code.

test.txt

```
Type: MESH
Name: GroundPlane
Location: [0.0, 0.0, 0.0]
Rotation: [0.0, 0.0, 0.0]
Scale: [1.0, 1.0, 1.0]
Diffuse: [0.8, 0.8, 0.8]
Specular: [1.0, 1.0, 1.0]
Shininess: 80.0
Reflectivity: 0.0
Transparency: 0.0
Refractive Index: 1.0
Texture: /Users/stephilyou/Documents/Fall Quarter 2025/Computer Graphics/Coursework/Textures/
texture1.ppm
Corner1: [-10.0, -10.0, 0.0]
Corner2: [10.0, -10.0, 0.0]
Corner3: [-10.0, 10.0, 0.0]
Corner4: [10.0, 10.0, 0.0]

Type: MESH
Name: Sphere_00
Location: [-1.066077709197998, 2.967869520187378, 1.5069036483764648]
Rotation: [3.504862070083618, 0.4511962831020355, 0.330922394990921]
Scale: [1.1168221235275269, 0.7043290734291077, 0.5309222936630249]
Diffuse: [0.7706326246261597, 0.6860113143920898, 0.3763737678527832]
Specular: [1.0, 1.0, 1.0]
Shininess: 145.0023059058681
Reflectivity: 0.0
Transparency: 0.0
Refractive Index: 1.0
```

Camera Space Transformation

In my code, the parseCameraInfo() parses and extracts the camera information based on how I formatted my txt file. In generateRay(), given a pixel, it computes a 3D ray that passes through the pixel. In generateAllRays(), it goes through a for loop to generate a ray for all the pixels in the camera's resolution. This creates one ray per pixel for the entire scene. For this code, I took inspiration from Lab 2 to complete it. To test and debug my code, I visualized the rays in Blender to make sure my code was correct. Instead of visualizing all of the rays which would be too computationally expensive given the resolution of the camera, I visualized 50 random rays as seen in the image below.

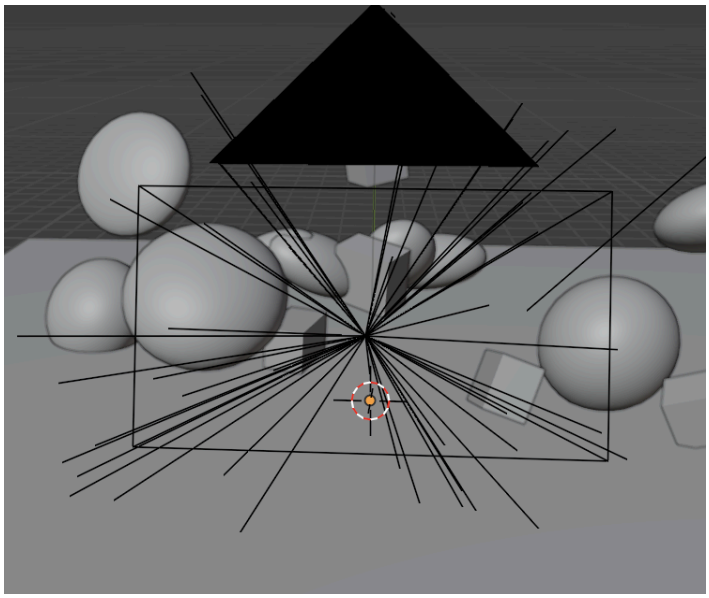
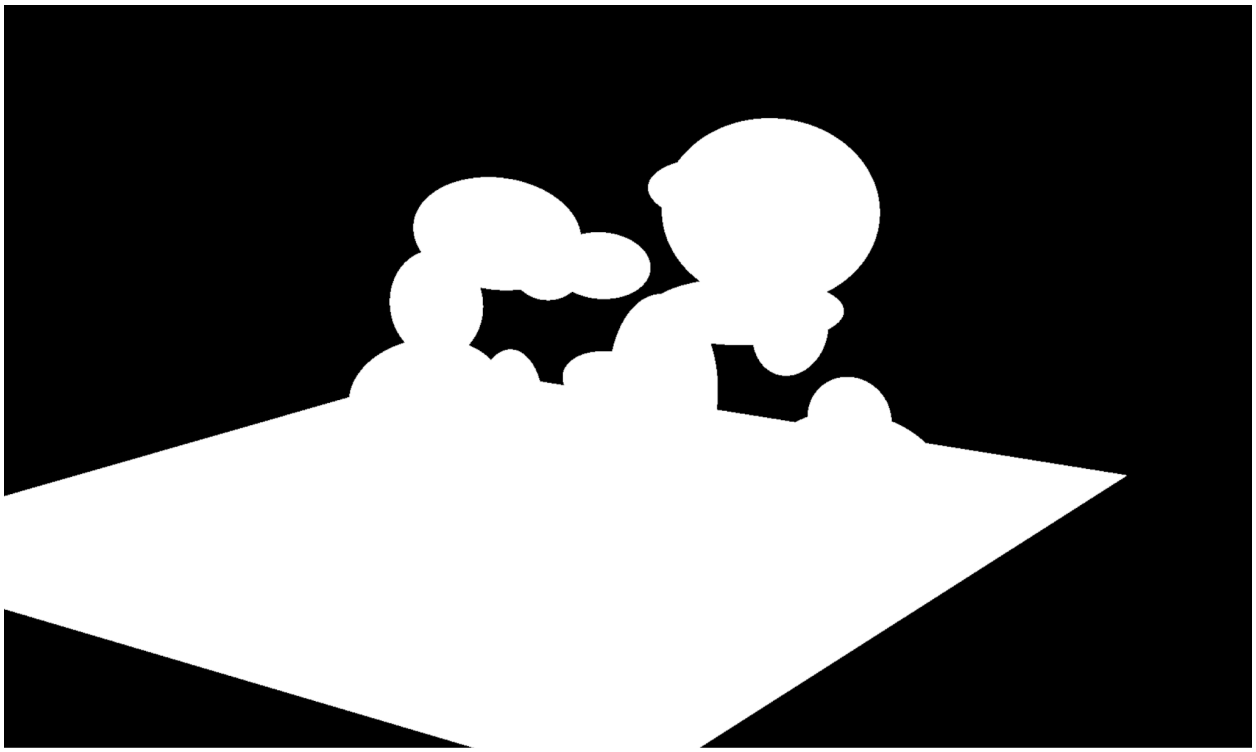


Image Read/Write

For my image read and write, I created an Image constructor and methods that sets the pixel color and saves a PPM file to render my output. I also have methods that get the pixel color, load a PPM file, and a bilinear texture sampling to smoothly filter the color for objects with texture. Since reading a texture PPM and writing a PPM file as the output is necessary in this project, evidence for completing this feature can be proven through my output images referenced below in the report.

Ray Intersection

In my ray intersection code, the header file contains all of my structs for the objects, hit, and material. In my cpp file, I parse through the txt file to obtain all of the information for my objects in the scene. For each object, I also have an intersect function that determines if the ray intersects with the object. I did use a coding assistant for this feature as I had difficulty when it came to transformed objects. Originally, my shapes were not accurate which was because I did not transform the ray before the intersection. The following image of Test1 demonstrates this feature. Before I implemented Module 3, I coded it such that a hit would be white and no hit would be black. From there, I was able to determine that if the shapes in my output were rendering like the scene, then my ray intersection code was correct.



Acceleration Hierarchy

I implemented a bounding volume hierarchy to improve the efficiency of ray intersections. I used a coding assistant to help me implement this feature. However, I did have to modify my code because I noticed that some of my shapes appeared “clipped” or “cut off” which meant that my bounding box was too tight, so it was missing some intersections. To fix this, I added an epsilon value to the min and max of the bounding box to slightly loosen it. The images below show my code and speedup for my BVH vs brute force using Test1.

```
(base) stephilyou@Stephis-MacBook-Pro Code % ./raytracer
Loaded 20 spheres, 1 planes, 0 cubes.
Rendered with BVH in 14041.7 ms (saved as output_bvh.ppm)
Rendered without BVH in 109781 ms (saved as output_nobvh.ppm)
Speedup (noBVH / BVH): 7.81825x
```

```
Image imgBVH(width, height);

auto t0 = std::chrono::high_resolution_clock::now();

for (int y = 0; y < height; ++y) {
    for (int x = 0; x < width; ++x) {
        Ray ray = generateRay(
            x, y,
            width, height,
            camera.sensorWidth,
            camera.sensorHeight,
            camera.focalLength,
            camera.location,
            camera.gazeVector,
            camera.upVector
        );

        Hit hit; hit.distance = 1e9;
        bool hitAny = bvhIntersectNode(bvhRoot.get(), ray, hit, prims, spheres, planes, cubes);
        Vec3 color = hitAny ? Vec3{1, 1, 1} : Vec3{0, 0, 0};
        imgBVH.setPixel(x, y, color);
    }
}

auto t1 = std::chrono::high_resolution_clock::now();
double msBVH = std::chrono::duration<double, std::milli>(t1 - t0).count();

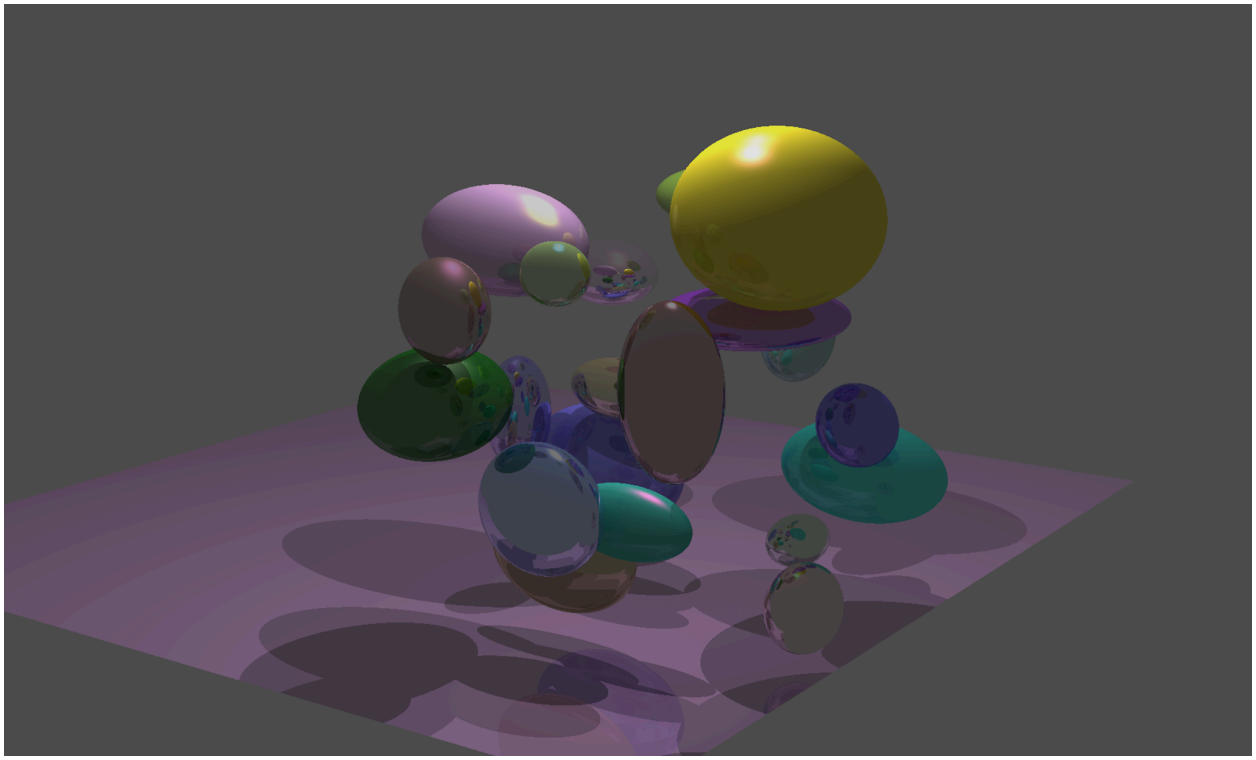
imgBVH.save("output_bvh.ppm", "P3");
cout << "Rendered with BVH in " << msBVH << " ms (saved as output_bvh.ppm)\n";
```

```
// Spheres
for (const auto &s : spheres) {
    Hit h;
    h.distance = closestHit.distance;
    if (s.intersect(ray, h) && h.distance < closestHit.distance) {
        closestHit = h;
    }
}
// Planes
for (const auto &p : planes) {
    Hit h;
    h.distance = closestHit.distance;
    if (p.intersect(ray, h) && h.distance < closestHit.distance) {
        closestHit = h;
    }
}
// Cubes
for (const auto &c : cubes) {
    Hit h;
    h.distance = closestHit.distance;
    if (c.intersect(ray, h) && h.distance < closestHit.distance) {
        closestHit = h;
    }
}

Vec3 color = closestHit.isHit ? Vec3{1, 1, 1} : Vec3{0, 0, 0};
imgNoBVH.setPixel(x, y, color);
```

Whitted-style Raytracing

I implemented Whitted-style Raytracing by creating a shading function which computes the color at an intersection using the diffuse and specular color of the object from the Blinn-Phong materials that I exported from Blender. I also have a function that casts shadow rays to each light to control how the lighting is. To implement reflecting and refracting, I used Snell's law and Schlick's approximation for Fresnel reflectance. I had a lot of difficulty with this feature, trying to get the reflections, lighting, shading, and coloring right. I used a coding assistant to implement this feature but had to change some code to adjust the lighting and coloring of my images. The image below shows the completed feature using Test1.

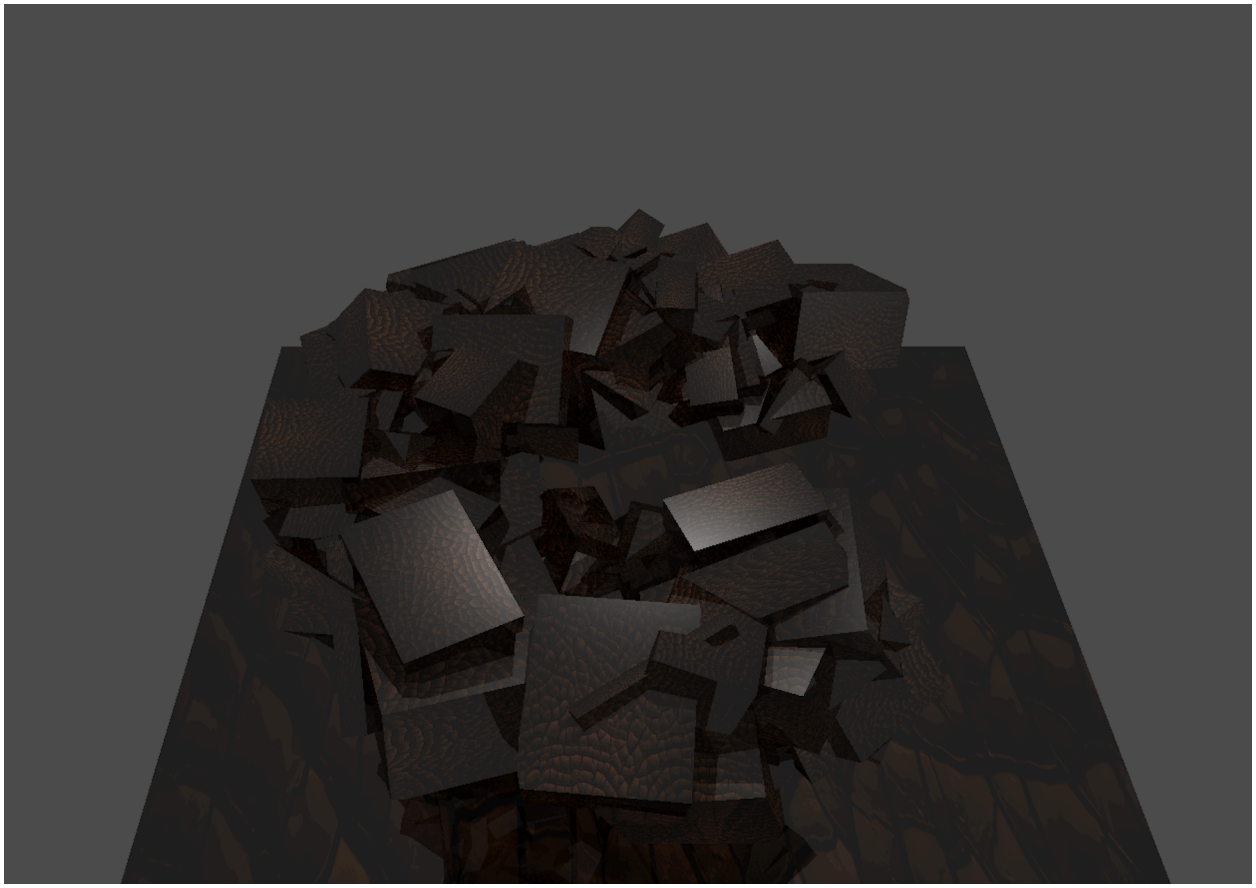


Antialiasing

I implemented adaptive supersampling antialiasing by tracing multiple rays at sub-pixel positions and only refining where the color varies a lot using recursion. This improves the efficiency of antialiasing and reduces runtime. I used a coding assistant to help me with this feature.

Textures

For textures, I created a Texture struct to store the width, height, and pixels. Then I created a load function to load the texture and return the texture ID which allows multiple textures in one scene. I have another function which looks up colors using UV coordinates. In my shading function, I modified it so that it fetches the texture image and uses that color. The image below shows the completed feature using Test2.

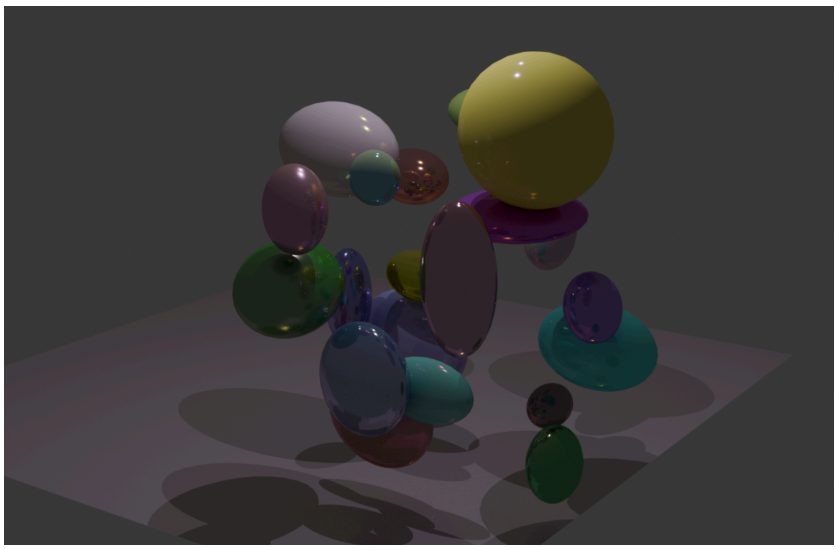


System Integration

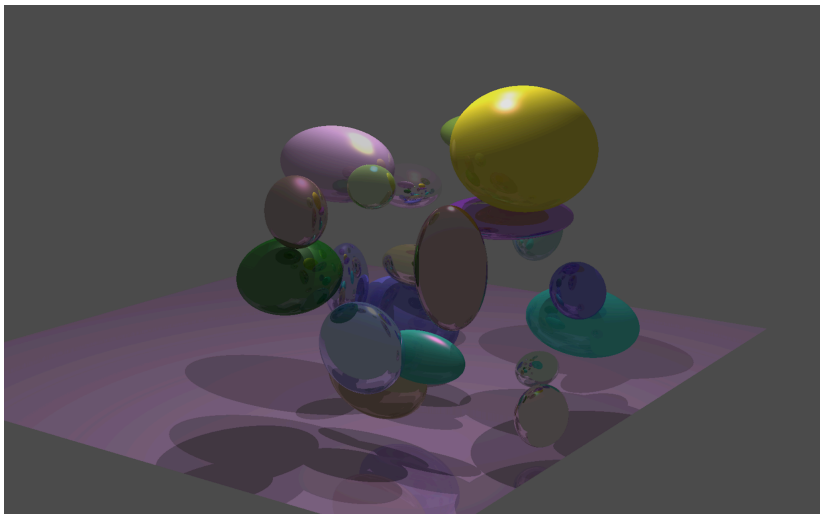
I integrated command lines to turn off features like acceleration hierarchy and antialiasing. In my system integration, you can also choose the parameters for antialiasing like max depth and samples per pixel.

Test Scenes

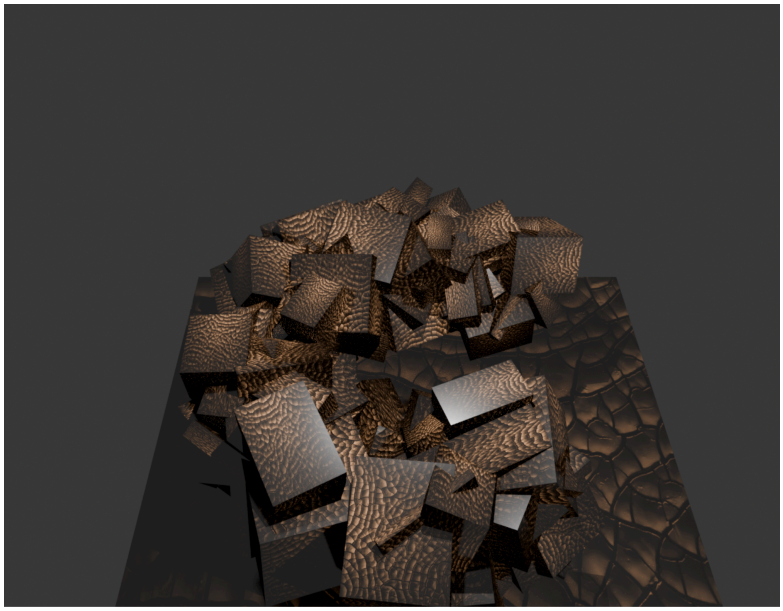
Blender:



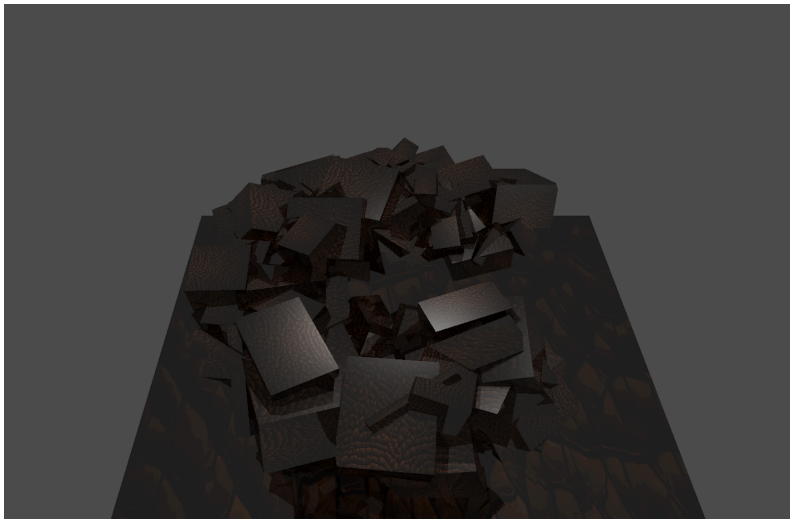
Mine:



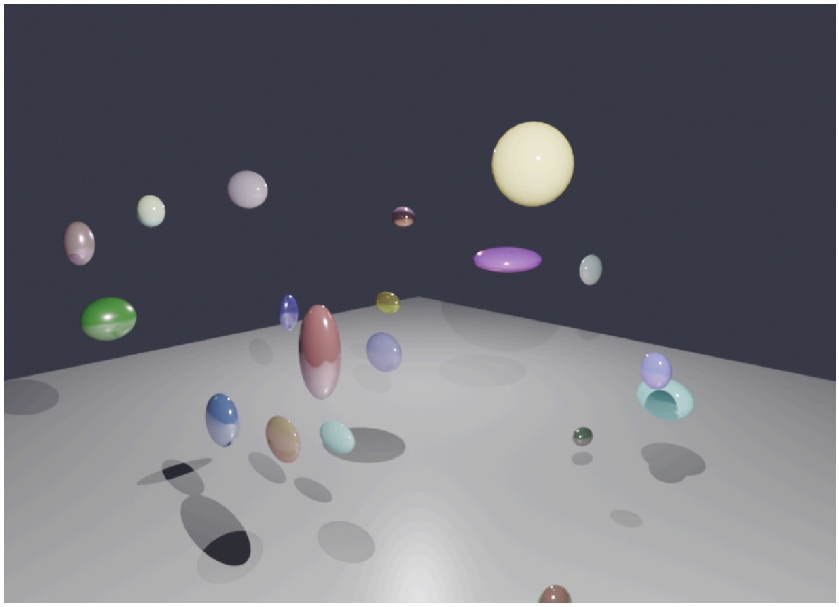
Blender:



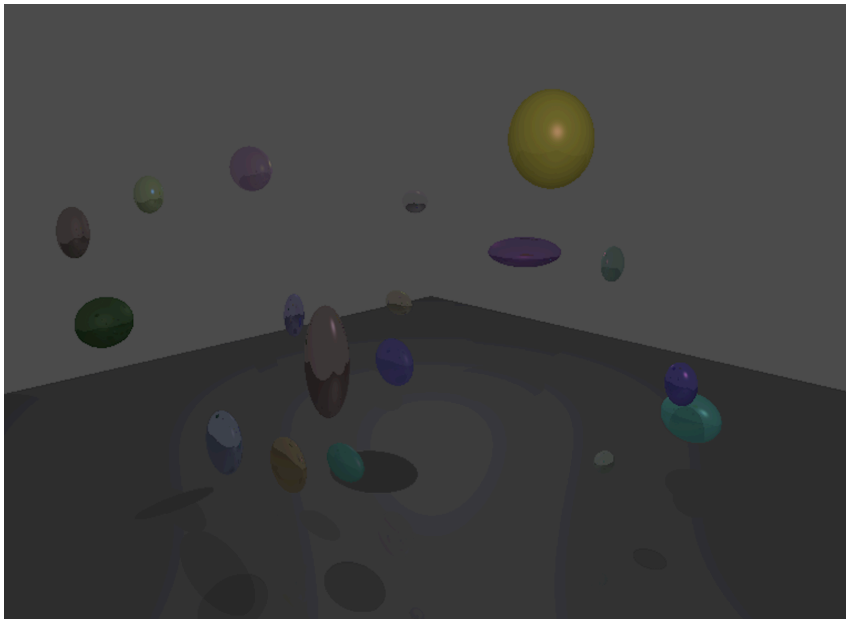
Mine:



Blender:



Mine:



Timeliness Bonus

- Module 1
    - I had to make some changes to Export.py file in order to export more information from the Blender scene like the Blinn-Phong and texture materials. For the camera space transformation, I did not change much of the code other than reformatting my file and parsing additional information from the Camera. For the image read/write, I had relatively the same code for loading and saving a PPM image. I did have the wrong interpretation of modifying pixels because I was

confused with the wording on the Coursework Specification, so I did have to change that part of my code. I would justify that I deserve the timeliness bonus because for the majority of my code, the implementation is the same. The only adjustments I made were to add additional code to implement features for the other modules or to reformat my code to be cleaner. As for modifying pixels, it was only one small portion of the entire module that I did not complete correctly, and therefore, I don't think I should be penalized for it. Once I started getting further through the project, I understood my mistake and fixed it by Module 2.

- Module 2
    - Again like the previous module, my code for the ray intersection and acceleration hierarchy did not change much other than adding more code to implement other features. For example, in my ray intersection code, I parse through all the objects, so I had to add code to parse through diffuse, specular, etc. For the acceleration hierarchy, I only added one other function which was to do intersection without BVH for my feature flag system integration. I would justify that I deserve the timeliness bonus because I did not heavily change my code to fix any mistakes. I only added to the code, so I could implement other features.
- Module 3
    - I did not submit anything for Module 3.

Exceptionalism

I implemented adaptive supersampling antialiasing which is a technique that allocates resources to regions of the image that exhibit high-frequency detail. Unlike uniform supersampling which traces additional rays for every pixel regardless of need, adaptive supersampling antialiasing selectively supersamples only pixels where aliasing is detected which significantly improves image quality while decreasing rendering times. I was motivated to use this method in order to improve the efficiency of my raytracer. It uses recursive subdivision to compare the color against its four neighbors. If the maximum color difference exceeds a threshold, then it returns the average. Otherwise, it recurses through each subregion.

Summary

I think a strength of the AI coding assistants is that it is good at handling documentation heavy aspects of Blender. For example, I used AI to help me learn how to traverse node trees and access material properties. As someone who wasn't familiar with Blender until I took this class, Blender can be an intimidating application to use at a first glance. AI was a great way for me to quickly learn how to navigate Blender. I think a major weakness of AI is that because this project is so large, it gets more difficult for AI to debug your code once you accumulate a lot of code files. Another major weakness is that AI doesn't understand the specific requirements of the ray

tracer implementation or how your data should be formatted. Again, because this assignment is so complex, it requires a lot of coordination between the Blender exporter and the ray tracer code to properly complete it.